

Complaint Intelligence Platform — Project Idea & Model

Oct 4, 2026 · @trishna

Part 1 — Project idea

A web platform where a support team manages customer complaints as tickets, and AI sorts, prioritises, routes and helps resolve each one. It is a complete enterprise product with an AI layer — not a chatbot, a single ML model or a plain dashboard.

The problem

- Support teams read and sort every complaint by hand, which is slow and inconsistent.
- Urgent complaints get buried in long queues and miss deadlines.
- Agents re-solve problems that were already solved in past tickets.
- Leadership has no real-time view of workload, deadline risk or recurring issues.

The solution

Every complaint becomes a trackable ticket, and AI assists at each decision point:

Complaint → Ticket → AI triage → Auto-routing → SLA deadline → Agent works with AI copilot → Resolution → Analytics

Users (2 roles)

Role	Responsibilities
Admin	Manages users, teams, SLA rules, categories and the knowledge base; sees the analytics dashboard; receives escalations and SLA-breach alerts; can do everything an agent can
Support Agent	Works on own and team tickets; uses AI triage, copilot, similar tickets and help articles; comments, changes status, escalates and resolves

End-to-end example

1. A customer writes: *"I was charged twice for my order of ₹12,500 and have already contacted support three times."*
2. Ticket **INC-10482** is created instantly with status **NEW**.
3. In the background, AI detects: category **Payments related**, sentiment **Very Negative**, amount **₹12,500**, priority **Critical**.
4. The ticket is routed to **Payments Support** and assigned to the least-busy agent.
5. A **2-hour SLA** starts with a live countdown.
6. The agent opens the ticket and sees an AI summary, likely root cause, similar past tickets, the refund-policy article, recommended actions and an editable draft reply.
7. At 80% of the SLA the agent is warned; at 100% the ticket auto-escalates and the admin is alerted.
8. The agent resolves the ticket; every step is logged.
9. The admin dashboard updates with volumes, SLA breach rate, workload and insights such as *"Refund complaints rose 43% this week."*

Key features

1. **Authentication and roles** — secure login with Admin and Agent permissions.
2. **Ticket management** — create, assign, escalate, resolve, reopen, comments, attachments, full history.
3. **AI triage** — category, intent, sentiment, entities, priority and confidence.
4. **AI copilot** — summary, root cause, next steps and draft reply, always approved by a human.
5. **Similar tickets and knowledge base (RAG)** — search by meaning; answers grounded in real data.
6. **SLA engine** — deadlines, countdowns, warnings and auto-escalation.
7. **Notifications** — real-time in-app alerts plus optional email.
8. **Analytics** — KPIs, trends, workload and emerging issues for the admin; "My stats" for agents.

Why it is strong

- A complete product: frontend, backend, database, event-driven processing and AI.
- Real enterprise features: role-based access, audit trails, SLAs and escalations.

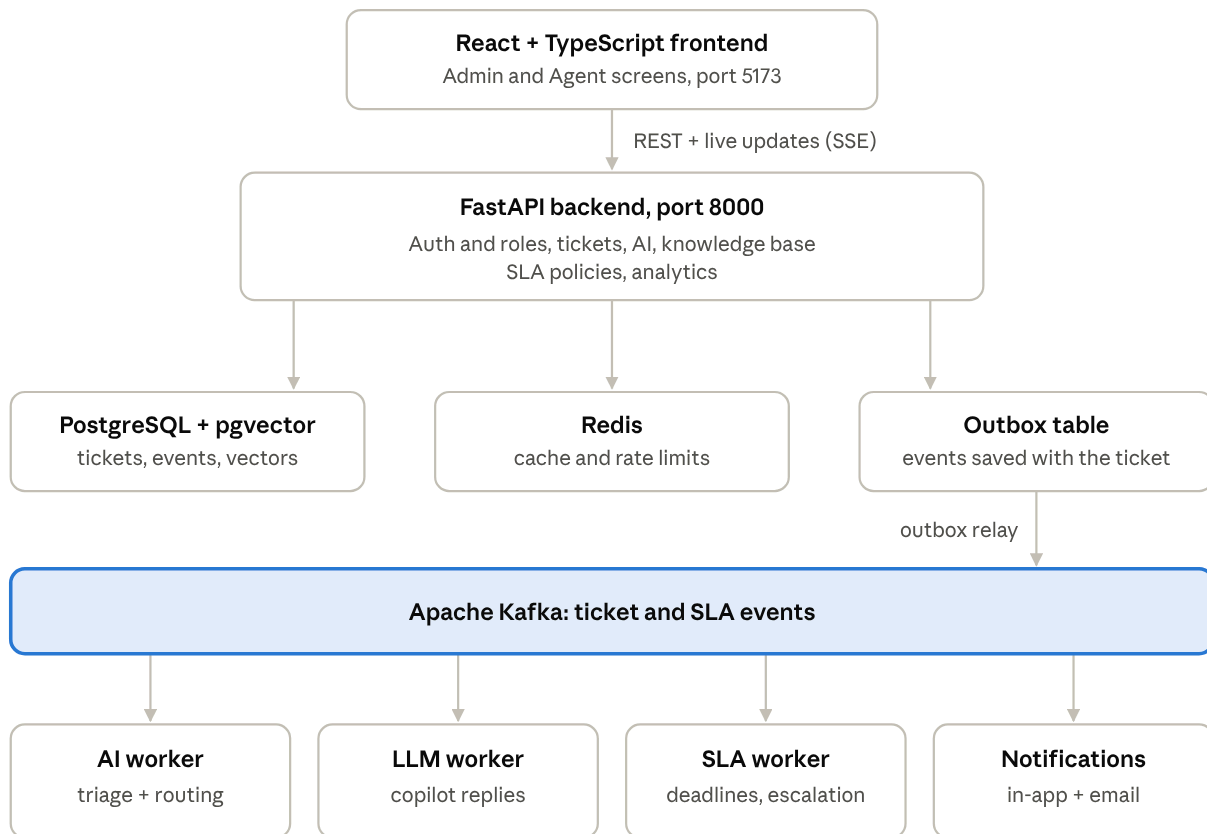
- Responsible AI: rules for critical decisions, AI for assistance, a human in the loop.
- A business analytics layer on top of operational data.
- Strong talking points for system design, ML, SQL and API interviews.

Part 2 — Project model

Architecture

The API writes tickets and events in one database transaction; Kafka then fans the events out to four background workers, so ticket creation stays instant while AI, SLA checks and notifications run asynchronously.

Ticket creation stays instant while four workers do the slow work



Workers save results to PostgreSQL; Prometheus and Grafana monitor every service

system architecture · API, data stores, Kafka, 4 workers

- **Outbox pattern** — events are saved with the ticket change, then a relay publishes them to Kafka, so none are lost.

- **Idempotent workers** — a processed_events table stops the same event being handled twice.
- **Retries and dead-letter queue** — failed events retry 3 times, then wait in a DLQ the admin can replay.

Technology stack

Layer	Technology
Frontend	React 18, TypeScript, Vite, Tailwind, React Query, React Router, Recharts
Backend	Python 3.12, FastAPI, SQLAlchemy 2 (async), Alembic, Pydantic v2
Auth	JWT access + rotating refresh tokens, argon2 hashing, role-based access
Database	PostgreSQL 16 + pgvector
Cache and rate limits	Redis
Messaging	Apache Kafka (KRaft), aiokafka, outbox pattern
ML	scikit-learn, Hugging Face Transformers (DistilBERT, sentiment), spaCy
Search	Sentence Transformers (MiniLM, 384 dims) + pgvector HNSW
GenAI	OpenAI API with structured outputs; a Mock provider so it runs without a key
Testing	Pytest, Vitest, React Testing Library, Playwright
DevOps	Docker Compose, GitHub Actions, Trivy, GHCR
Monitoring	Prometheus, Grafana, structured JSON logs
Cloud (optional)	AWS EC2 (simple) or ECS + RDS + S3 (advanced)

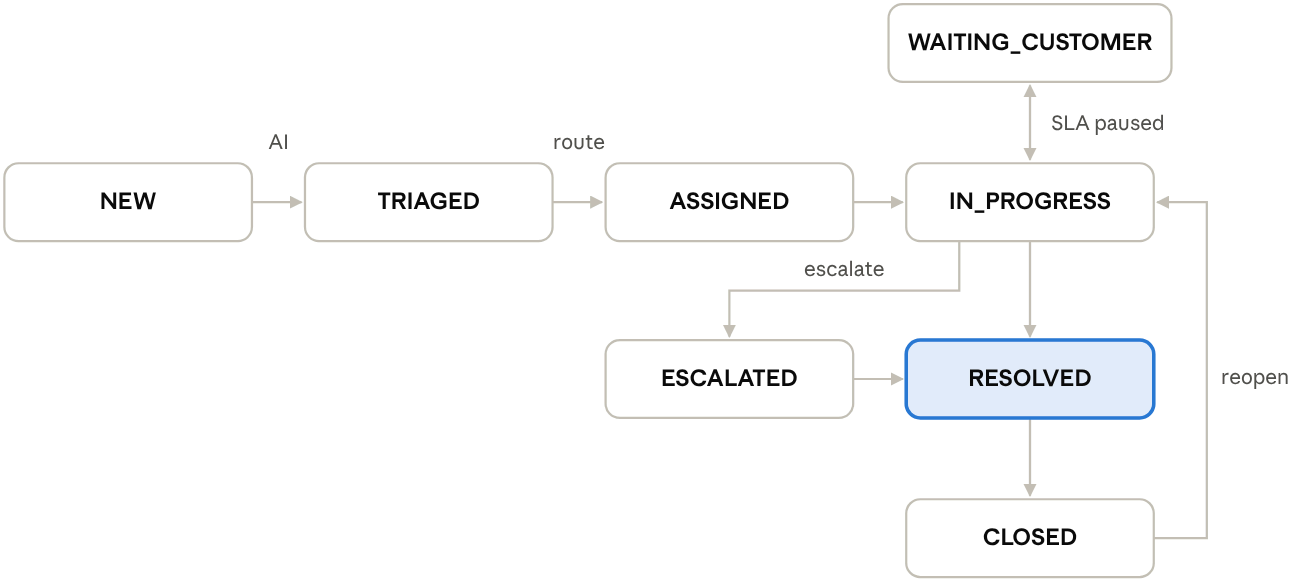
Roles and permissions

Action	Agent	Admin
View and work on own/team tickets	Yes	Yes
View all tickets	No	Yes
Create tickets, comment, change status, resolve	Yes	Yes
Escalate a ticket	Yes	Yes
Reassign outside own team, close others' tickets	No	Yes
Use AI copilot, similar tickets, KB search	Yes	Yes
Create and edit knowledge articles	No	Yes
Manage users, teams, SLA policies, categories	No	Yes
Analytics dashboard	My stats only	Yes
Receive escalation and SLA-breach alerts	No	Yes
Replay failed events (DLQ)	No	Yes

Ticket lifecycle

A ticket moves NEW → TRIAGED → ASSIGNED → IN_PROGRESS, may wait on the customer or be escalated, and ends RESOLVED → CLOSED, with reopen possible.

Every ticket follows one path from NEW to CLOSED



ticket lifecycle · 8 states

- Allowed moves are defined in one place; invalid moves are rejected (HTTP 409).
- Every change is written to the ticket timeline; sensitive actions also go to the audit log.
- The SLA clock pauses while the ticket is WAITING_CUSTOMER.

Database model

Table	Key fields
users	id, name, email, password_hash, role (ADMIN/AGENT), team_id, created_at
departments	id, name
teams	id, department_id, name (e.g. Payments Support)
customers	id, customer_code, name, segment, region, created_at
categories	id, name, description

Table	Key fields
tickets	id, ticket_number (INC-xxxxx), customer_id, subject, description, description_source, channel, product, order_id, amount_inr, city, category, intent, sentiment, priority, status, assignee_id, team_id, csat_score, created_at, first_response_at, updated_at, resolved_at
ticket_comments	id, ticket_id, author_id, body, ai_assisted, created_at
ticket_events	id, ticket_id, event_type, actor_id, metadata, created_at
ai_analyses	id, ticket_id, category, intent, sentiment, priority, entities, summary, root_cause, recommendations, draft_response, confidence, model_version, prompt_version, created_at
sla_policies	id, priority, category, target_minutes
sla_events	id, ticket_id, event_type (started/warning/breached/paused), deadline, created_at
knowledge_articles	id, title, body, category, embedding (vector 384), usage_count, created_at, updated_at
ticket_embeddings	id, ticket_id, embedding (vector 384), model_name, created_at
notifications	id, user_id, type, title, message, read_at, created_at
audit_logs	id, actor_id, action, resource_type, resource_id, metadata, created_at
outbox	id, event_type, payload, published_at
processed_events	consumer, event_id
refresh_tokens	id, user_id, token_hash, expires_at, revoked

Kafka events

Event	Produced when	Consumed by
ticket.created	A ticket is submitted	AI worker
ai.analysis.completed	Triage finishes	LLM worker, SLA worker
ticket.assigned	A ticket is routed or reassigned	Notification worker
ticket.updated / ticket.resolved	Status changes	Analytics, embeddings
ticket.escalated	Manual or automatic escalation	Notification worker (alerts admin)
sla.warning	80% of SLA time used	Notification worker (alerts agent)
sla.breached	100% of SLA time used	Auto-escalation, alerts admin

Each event carries event_id, type, timestamp, ticket_id, actor and payload.

AI / ML model

Each task uses the tool that fits it; the LLM assists, while rules make the critical decisions.

Task	Approach	Trained on or validated with
Category (12 classes, e.g. Returns, Refund Related, Payments related)	TF-IDF + Logistic Regression baseline, then a text + structured-fields model, then DistilBERT if it scores better	Dataset category column
Intent	Same pipeline	Dataset Sub-category column
Sentiment (5 levels)	Pretrained Hugging Face model	Agreement with CSAT score
Entities (order IDs, ₹ amounts, dates, products)	spaCy + regex rules	Fixed test examples

Task	Approach	Trained on or validated with
Priority (Low, Medium, High, Critical)	Business rules on category, sentiment, amount and repeat contact; rules can only raise priority	Documented rules (no labels in the data)
Similar tickets and KB search	MiniLM embeddings + pgvector cosine search, plus keyword search	Fixed test examples
Copilot (summary, root cause, actions, draft reply)	OpenAI structured JSON, grounded in similar tickets and KB articles (RAG), PII masked first	Schema validation and manual review
Routing and SLA	Deterministic rules	Unit tests

Example triage output (the confidence is illustrative; real numbers come from the evaluation reports):

```
{
  "category": "Payments related",
  "intent": "Duplicate payment",
  "sentiment": "Very Negative",
  "priority": "Critical",
  "entities": {"amount": "₹12,500"},
  "confidence": 0.91,
  "model_version": "triage-v1"
}
```

Responsible AI rules

- AI never sends a reply on its own; an agent must review and accept it.
- Personal data is masked before any text goes to the LLM.
- Every AI output stores its confidence and model version.
- Priority, SLA and routing use rules, never the LLM.

Dataset

[Kaggle “eCommerce Customer Service Satisfaction”](#) holds 85,907 real support records from an Indian e-commerce company; it is the only dataset used and is not committed to GitHub.

Column	Used as
category / Sub-category	Category and intent labels
Customer Remarks	Complaint text (present in about 33% of rows)
channel_name	Channel (Inbound, Outcall, Email)
Customer_City, Product_category, Item_price	Region, product and ₹ amount (about 20% filled)
order_date_time, Issue_reported at, issue_responded	Real timestamps and first-response time
Agent_name, Supervisor, Manager	Seed agents and team names
CSAT Score	Satisfaction metric and sentiment validation

Known limitations

- Remarks are short or empty; empty ones get templated descriptions that are excluded from training.
- There are no priority labels, so priority is rule-based.
- City and product data is partial, so those charts show their data coverage.

API endpoints

All endpoints sit under `/api/v1/`.

Area	Endpoints
Auth	POST <code>/auth/login</code> , <code>/auth/refresh</code> , <code>/auth/logout</code> · GET <code>/auth/me</code>

Area	Endpoints
Tickets	POST/GET /tickets · GET/PATCH /tickets/{id} · POST /tickets/{id}/assign, /escalate, /resolve, /close, /reopen, /comments
AI	POST /ai/analyze · POST /ai/draft-response
Retrieval	GET /tickets/{id}/similar · GET /knowledge/search · CRUD /knowledge
SLA	CRUD /sla-policies (admin)
Notifications	GET /notifications · POST /notifications/{id}/read · live SSE stream
Analytics (admin)	GET /analytics/overview, /trends, /sla, /categories, /workload, /emerging
Admin	CRUD /users, /teams, /categories · GET/POST /admin/dlq

Screens

Screen	Admin	Agent
Login	Yes	Yes
Ticket Queue (search, filters)	All tickets	Own and team
Create Ticket	Yes	Yes
Ticket Details (main workspace)	Yes	Yes
My Work + My stats	Yes	Yes
Knowledge Base	Read and edit	Read
Notifications	Yes	Yes
Admin Dashboard + Analytics	Yes	No
Admin: Users, Teams, SLA Policies, Categories	Yes	No

The Ticket Details page is the agent's main workspace. It shows:

- the original complaint, customer profile and previous tickets;
- AI category, intent, sentiment, priority and confidence;
- the SLA countdown badge (green, amber, red);
- the AI summary, extracted entities and recommended actions;
- similar past tickets and relevant help articles;
- an editable AI draft reply (accept, regenerate, discard);
- comments, the full event timeline, and buttons to assign, escalate, change status, resolve and reopen.

Analytics

The admin dashboard tracks:

- ticket volume by day, week and month;
- rates by channel, city and product;
- open and critical tickets;
- SLA breach rate;
- average resolution time and first-response time;
- repeat-complaint rate;
- agent and team workload;
- sentiment and CSAT trends;
- top categories and intents;
- emerging issues (week-over-week growth);
- an auto-generated plain-English insight.

Agents see "My stats" instead: open tickets, tickets with SLA at risk, and tickets resolved this week.

Security and reliability

- JWT authentication with rotating refresh tokens, argon2 password hashing and role checks on every endpoint.
- Input validation (Pydantic), Redis rate limiting, CORS and security headers.
- Audit logging for sensitive actions; PII masking before LLM calls.
- Secrets kept in .env only, never committed.
- Outbox pattern, idempotent consumers, retries and a dead-letter queue.

- Healthchecks, structured logs, Prometheus metrics and Grafana dashboards.

Testing

- **Unit tests** — state machine, SLA math (including the pause rule), priority rules, PII masking, entity extraction.
- **API tests** — authentication and Admin vs Agent permissions.
- **Integration tests** — Kafka flow, idempotency, retry and dead-letter queue.
- **ML evaluation** — precision, recall, F1 and confusion matrix on a held-out test set.
- **Frontend tests** — components with Vitest.
- **End-to-end test (Playwright)** — complaint → triage → assignment → SLA → resolution → dashboard.

Repository structure

```
complaint-intelligence/
├── frontend/           React app
├── backend/app/
│   ├── api/           routes
│   ├── models/        database tables
│   ├── schemas/       request/response shapes
│   ├── repositories/  database queries
│   ├── services/      business logic
│   ├── ai/            triage, copilot, embeddings
│   ├── workers/       AI, LLM, SLA, notification workers
│   ├── events/        Kafka, outbox
│   ├── auth/          JWT, roles
│   └── core/          config, logging, errors
├── ml/                training scripts, reports, artifacts
├── data/              dataset (not committed)
├── tests/
├── docker/
├── docs/              data profile, deployment guide
├── scripts/           dev.ps1
├── .github/workflows/ CI/CD
├── docker-compose.yml
├── Makefile
├── PROGRESS.md
└── README.md
```

Build plan

1. **Setup** — folders, .gitignore, dataset profile, first push.
2. **Foundation** — Docker, database schema, FastAPI, React shell, login with 2 roles.
3. **Ticket engine** — ticket workflow, permissions, CSV import, ticket screens.
4. **AI triage** — data profiling, classifiers, sentiment, entities, priority rules, evaluation.
5. **Events** — Kafka, outbox, AI worker, routing, retries, dead-letter queue.
6. **Copilot** — LLM summaries, actions, draft replies, PII masking.
7. **Retrieval** — embeddings, similar tickets, knowledge base, RAG.
8. **SLA and notifications** — deadlines, countdowns, escalation, alerts, email.
9. **Analytics** — admin dashboard, KPIs, trends, insights, My stats.
10. **Production** — tests, monitoring, CI/CD, security, deployment docs, final README.